QA AUTOMATION · AI-ASSISTED TESTING

AI-Assisted Test Automation

High-Level Approach

Prepared by: Chakri, QA Automation · November 2026

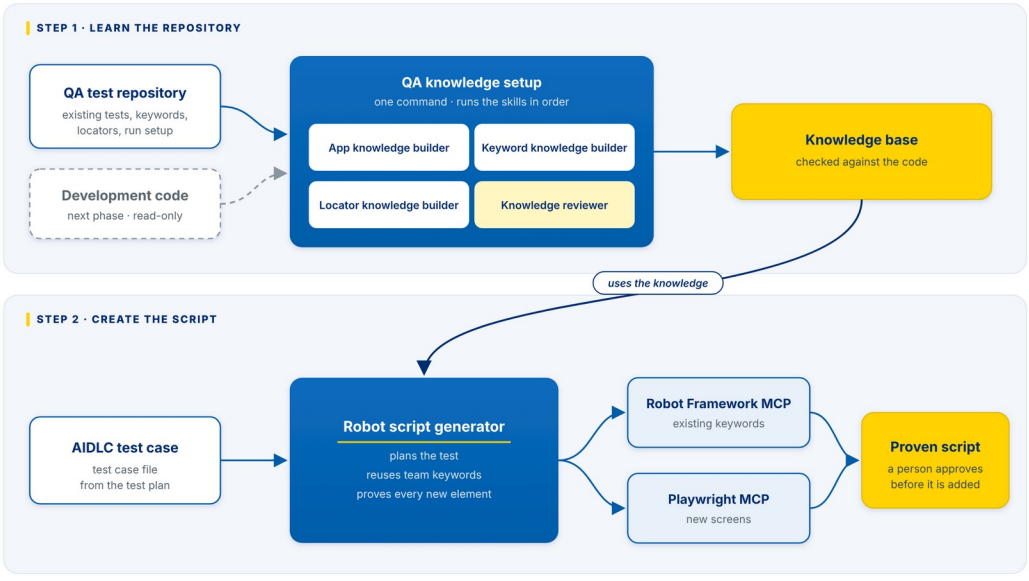
AIDLC takes us from a requirement to test cases, but it does not create Robot Framework test scripts. **We have built our own AI skills to close that gap:** they learn our test automation first, then turn AIDLC test cases into working, proven Robot Framework scripts.

1. Where this fits

1. **Requirement:** the business requirement and design documents (for example, the HLD).
2. **Test plan:** created with AIDLC from the requirement.
3. **Test cases / scenarios:** created with AIDLC from the test plan, saved as a test case file.
4. **Test scripts (our skills):** Robot Framework scripts created from those test cases. AIDLC does not cover this step, so this is what we built.

2. Our approach

To write a reliable script for a new feature, the AI must first understand our existing automation, the same way a new team member would. So the approach has two steps.



How the skills fit together. The dashed box is the next phase.

Step 1: Learn the repository (knowledge skills)

Skill	What it does
QA knowledge setup	The single command that runs all the skills below in the right order and builds the knowledge base.
App knowledge builder	Learns the application: screens, business journeys, an application map showing how screens connect, business terms, and how tests are run.
Keyword knowledge builder	Lists the ready-made steps (keywords) the team has already built, what each one does and which business step it covers.
Locator knowledge builder	Lists where each button and field sits on screen, and how reliable each reference is.
Knowledge reviewer	Checks every entry against the real code before the knowledge is trusted.

The knowledge can be built for one suite, several suites, or the whole repository, and refreshed when tests change.

Step 2: Create the script (Robot script generator)

1. It reads the AIDLC test case file and the knowledge base.
2. It plans the test: which steps already exist as team keywords, and which are new.
3. Using **Robot Framework MCP**, it drives the real application with the team's existing keywords to reach the right screen.
4. Using **Playwright MCP**, it looks at the new screen and finds the new fields and buttons.
5. It proves every new element, runs the full test on screen, and reports the result.
6. A person reviews and approves before anything is added to the team's tests.

3. What we have achieved so far

- Knowledge skills built and reviewed for our pilot suites.
- Robot script generator working end to end in UAT.
- **Mock screen support:** for features not yet released, it builds the test from a mock screen, checks the route on the real application, and holds the real test until release.

> **Pilot result**
>
> From one AIDLC test case, it produced a passing mock test, a passing route check on UAT, and the real test ready for release. It also flagged outdated screen references and slow steps for the team.

4. What we are adding next

Development repository knowledge. Today, for an unreleased feature, the AI needs a mock screen. Next, it will also learn the development code (read-only): each screen, its fields and buttons, validation rules and who can see what. When a tester asks for a script for a new screen, it will combine:

- the **development knowledge**, for the new screen, and
- the **existing keyword knowledge**, for everything already automated.

Mock screens then become a fallback, not a requirement. The AI will use the code version that is actually deployed to the environment we test.

Other improvements planned:

- **Faster, lower-cost knowledge building:** a script handles the exact facts, the AI adds only the meaning, and unchanged files are skipped.
- **Early warnings:** when developers change a screen, it flags which tests will be affected before they fail.
- **Central access:** one shared copy of the skills, so the whole QA team uses the same version in Devin or VS Code.

5. How we keep the scripts reliable

- Existing team keywords are always reused; new ones are created only for new behaviour.
- Every new element is checked on a running screen before it is used.
- A test is only reported as passed after a real Robot Framework run.
- The AI never changes the team's files; it suggests fixes and a person approves.
- Passwords come only from the existing secure vault and are never shown or stored.